Secure Azure Banking Prototype with Bicep

1. Azure Bicep Configuration

File: main.bicep

```
targetScope = 'resourceGroup'

param location string = resourceGroup().location
param prefix string = 'banklab'
var suffix = uniqueString(subscription().subscriptionId, resourceGroup().id)

resource nsg 'Microsoft.Network/networkSecurityGroups@2024-05-01' = {
 name: '${prefix}-app-nsg'
 location: location
 properties: { securityRules: [{
  name: 'Deny-Internet-Inbound'
  properties: {
   priority: 4096
   direction: 'Inbound'
   access: 'Deny'
   protocol: '*'
   sourceAddressPrefix: 'Internet'
   sourcePortRange: '*'
   destinationAddressPrefix: '*'
   destinationPortRange: '*'
  }
 }]}
}

resource vnet 'Microsoft.Network/virtualNetworks@2024-05-01' = {
 name: '${prefix}-vnet'
 location: location
 properties: {
  addressSpace: { addressPrefixes: ['10.40.0.0/16'] }
  subnets: [
   { name: 'snet-app'; properties: { addressPrefix: '10.40.1.0/24'; networkSecurityGroup: { id: nsg.id } } }
   { name: 'snet-private-endpoints'; properties: { addressPrefix: '10.40.2.0/24';
privateEndpointNetworkPolicies: 'Disabled' } }
  ]
 }
}

resource peSubnet 'Microsoft.Network/virtualNetworks/subnets@2024-05-01' existing = {
 name: 'snet-private-endpoints'
 parent: vnet
}
```

```
resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: toLower('${prefix}${suffix}')
  location: location
  sku: { name: 'Standard_GRS' }
  kind: 'StorageV2'
  properties: {
    allowBlobPublicAccess: false
    allowSharedKeyAccess: false
    minimumTlsVersion: 'TLS1_2'
    publicNetworkAccess: 'Disabled'
    supportsHttpsTrafficOnly: true
  }
}

resource vault 'Microsoft.KeyVault/vaults@2024-11-01' = {
  name: toLower('${prefix}-${suffix}-kv')
  location: location
  properties: {
    tenantId: tenant().tenantId
    enableRbacAuthorization: true
    enablePurgeProtection: true
    enableSoftDelete: true
    softDeleteRetentionInDays: 90
    publicNetworkAccess: 'Disabled'
    sku: { family: 'A'; name: 'standard' }
  }
}

resource storagePe 'Microsoft.Network/privateEndpoints@2024-05-01' = {
  name: '${prefix}-storage-pe'
  location: location
  properties: {
    subnet: { id: peSubnet.id }
    privateLinkServiceConnections: [{ name: 'storage-blob'; properties: { privateLinkServiceId: storage.id;
groupIds: ['blob'] } }]
  }
}

resource vaultPe 'Microsoft.Network/privateEndpoints@2024-05-01' = {
  name: '${prefix}-vault-pe'
  location: location
  properties: {
    subnet: { id: peSubnet.id }
    privateLinkServiceConnections: [{ name: 'key-vault'; properties: { privateLinkServiceId: vault.id; groupIds:
['vault'] } }]
  }
}

output virtualNetworkName string = vnet.name
```

```
output storageAccountName string = storage.name
output keyVaultResourceName string = vault.name
```

2. Reflective Report

For this exercise, I used Microsoft Azure Bicep, a declarative infrastructure-as-code language, to model a small banking workload rather than configuring resources manually through the portal. The design creates a virtual network with separate application and private-endpoint subnets, a network security group, an encrypted storage account, and an Azure Key Vault. Both data services disable public network access and use private endpoints. The configuration therefore demonstrates Infrastructure as a Service networking, managed platform services, repeatable deployment, and security-by-design in one compact artefact. This approach reflects cloud computing's defining characteristics of on-demand provisioning, resource pooling, elasticity, measured service, and network accessibility (Mell and Grance, 2011).

I personalised the design around my experience leading cyber and information security in an African banking group. In that context, a technically functional deployment is insufficient; architecture must also support confidentiality, traceability, resilience, data-residency assessment, and controlled administrative access. I therefore treated the script as a laboratory prototype for a non-production banking application and deliberately excluded customer data, credentials, and live subscription identifiers. Key Vault

represents centralised protection of secrets, while private endpoints reduce exposure to the public internet. Bicep's declarative syntax also made the intended state easier to review than a sequence of portal screenshots. Microsoft recommends managed identities where possible, secure parameters where secrets are unavoidable, and avoiding secret values in deployment outputs (Microsoft, 2025a).

The main positive experience was repeatability. Variables, parameters, dependencies, and outputs converted architectural intent into a version-controlled specification. The same pattern could be promoted across development, test, and production with environment-specific parameters and approval gates. This supports consistency and reduces configuration drift, although automation does not guarantee correctness. Research on infrastructure-as-code defects shows that scripts can contain security and maintainability weaknesses, making peer review and automated testing necessary (Rahman et al., 2019).

Several challenges emerged. First, Azure resource providers use different API versions and property schemas; a template that looks syntactically coherent may still fail validation or behave differently after service evolution. Second, private networking introduces dependencies among subnets, DNS zones, private endpoints, and access policies. The short script creates endpoints but omits private DNS zones to remain concise, so name resolution would require further configuration before production use. Third, banking

controls extend beyond deployment. The model does not yet implement customer-managed keys, diagnostic settings, immutable audit retention, backup, disaster recovery, Azure Policy assignments, Defender for Cloud, privileged identity management, or connectivity to a security operations centre. Azure's Key Vault resource documentation confirms that Bicep exposes controls including RBAC authorisation, soft deletion, retention, and public-network access (Microsoft, 2026).

Overall, the exercise deepened my understanding that cloud implementation combines service-model knowledge with governance engineering. I would next modularise the code, add private DNS, managed identities, least-privilege role assignments, policy-as-code, logging, and continuous Bicep linting in a controlled CI/CD pipeline. Before any banking deployment, the design would require threat modelling, architecture review, regulatory assessment, cost modelling, resilience testing, and formal change approval. This reinforced that secure cloud adoption requires disciplined collaboration among security, architecture, operations, risk, compliance, and business teams throughout.

3. References

Mell, P., & Grance, T. (2011). The NIST definition of cloud computing (NIST SP 800-145). NIST. https://doi.org/10.6028/NIST.SP.800-145

Microsoft. (2025a). Use Bicep to manage secrets. Microsoft Learn. https://learn.microsoft.com/azure/azure-resource-manager/bicep/scenarios-secrets

Microsoft. (2025b). What is Bicep? Microsoft Learn. https://learn.microsoft.com/azure/azure-resource-manager/bicep/overview

Microsoft. (2026). Microsoft.KeyVault/vaults: Bicep resource definition. Microsoft Learn. https://learn.microsoft.com/azure/templates/microsoft.keyvault/vaults

Rahman, A., Mahdavi-Hezaveh, R., & Williams, L. (2019). A systematic mapping study of infrastructure as code research. Information and Software Technology, 108, 65-77. https://doi.org/10.1016/j.infsof.2018.12.004